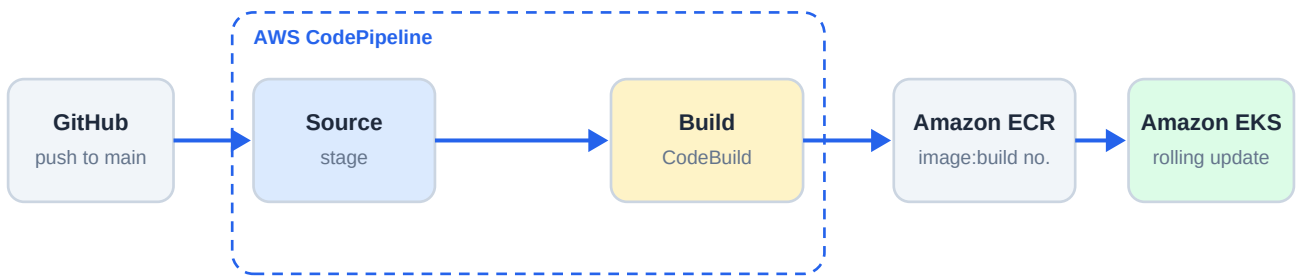


DevOpsMatrix CI/CD Guide

CodePipeline, CodeBuild, Amazon ECR and Amazon EKS, set up from scratch. Written for new team members. Region: ap-south-1.



CodeBuild builds the image, pushes it to ECR, then deploys to EKS with `deploy.sh`

Figure 1. End-to-end flow. There is no separate Deploy stage; deployment happens inside CodeBuild.

What each piece does

Service	Job in this project
CodePipeline	Watches the GitHub main branch and starts a run on every push.
CodeBuild	Runs <code>buildspec.yml</code> : build the Docker image, push to ECR, deploy to EKS.
Amazon ECR	Stores the images, tagged with the CodeBuild build number.
Amazon EKS	Runs the app. <code>deploy.sh</code> applies the manifests and swaps in the new image.

Why no CodeDeploy? CodeDeploy is for EC2, Lambda and ECS. For EKS the standard approach is `kubectl`, which is what `deploy.sh` uses. So the pipeline has only two stages: Source and Build.

Prerequisites

- A GitHub repo (admin access) and an AWS account that can create IAM roles, CodeBuild, CodePipeline and ECR resources.
- An existing EKS cluster named `eksCluster`, and the AWS CLI and `kubectl` installed locally for one-time setup.
- A `Dockerfile` in the repo root.
- Use one Region (`ap-south-1`) for everything. Resources in other Regions will not be found.

Step 1: Repository layout

```
repo/
|-- buildspec.yml
|-- Dockerfile
'-- EKS/kubernetes/
    |-- namespace.yaml
    |-- deployment.yaml
    |-- service.yaml
    -- deploy.sh
```

Step 2: Kubernetes manifests and deploy script

The container in `deployment.yaml` must be named `nginx`, because the script runs `kubectl set image ... nginx=...`. Each YAML document goes in its own file.

```

# namespace.yaml
apiVersion: v1
kind: Namespace
metadata: {name: static-site}
---
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: static-site, namespace: static-site}
spec:
  replicas: 2
  selector: {matchLabels: {app: static-site}}
  template:
    metadata: {labels: {app: static-site}}
    spec:
      containers:
        - name: nginx
          image: nginx:stable # replaced at deploy time with the ECR image
          ports: [{containerPort: 80}]
---
# service.yaml
apiVersion: v1
kind: Service
metadata: {name: static-site, namespace: static-site}
spec:
  type: LoadBalancer
  selector: {app: static-site}
  ports: [{port: 80, targetPort: 80}]

```

A minimal EKS/kubernetes/deploy.sh. It receives EKS_CLUSTER, AWS_REGION and IMAGE_URI from the buildspec. Use your team's own version if you already have one.

```

#!/bin/sh
set -eu
NS=static-site

# Install kubectl only if the build image does not have it
if ! command -v kubectl >/dev/null 2>&1; then
  V=$(curl -sL https://dl.k8s.io/release/stable.txt)
  curl -sLO "https://dl.k8s.io/release/$V/bin/linux/amd64/kubectl"
  chmod +x kubectl && mkdir -p "$HOME/bin" && mv kubectl "$HOME/bin/"
  export PATH="$HOME/bin:$PATH"
fi

aws eks update-kubeconfig --region "$AWS_REGION" --name "$EKS_CLUSTER"
kubectl get nodes

kubectl apply -f EKS/kubernetes/namespace.yaml
kubectl apply -f EKS/kubernetes/deployment.yaml
kubectl apply -f EKS/kubernetes/service.yaml

kubectl set image deployment/static-site nginx="$IMAGE_URI" -n "$NS"
kubectl rollout status deployment/static-site -n "$NS" --timeout=180s
kubectl get deployment,pods,service -n "$NS"

```

Step 3: The buildspec

Save this as buildspec.yml in the repo root. Three phases, plus two variables at the bottom. Long lines are wrapped in this PDF for printing; copy the original from the repository.

```

version: 0.2

phases:
  pre_build:
    commands:
      - AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
      - ECR_REGISTRY=${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_DEFAULT_REGION}.amazonaws.com
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION | docker login --username AWS --password-stdin $ECR_REGISTRY
      - |
        if aws ecr describe-repositories --region $AWS_DEFAULT_REGION \
          --repository-names $ECR_REPOSITORY >/dev/null 2>&1
        then echo "ECR repository already exists."
        else aws ecr create-repository --region $AWS_DEFAULT_REGION --repository-name $ECR_REPOSITORY
        fi

```

```

build:
  commands:
    - docker build --pull -t $ECR_REGISTRY/$ECR_REPOSITORY:$CODEBUILD_BUILD_NUMBER .
    - docker image inspect $ECR_REGISTRY/$ECR_REPOSITORY:$CODEBUILD_BUILD_NUMBER

post_build:
  commands:
    - docker push $ECR_REGISTRY/$ECR_REPOSITORY:$CODEBUILD_BUILD_NUMBER
    - aws ecr describe-images --region "$AWS_DEFAULT_REGION" --repository-name "$ECR_REPOSITORY" --image-ids
      "imageTag=$CODEBUILD_BUILD_NUMBER"
    - EKS_CLUSTER="$EKS_CLUSTER" AWS_REGION="$AWS_DEFAULT_REGION"
      IMAGE_URI="$ECR_REGISTRY/$ECR_REPOSITORY:$CODEBUILD_BUILD_NUMBER" sh EKS/kubernetes/deploy.sh
    - echo "Build completed successfully."

env:
  variables:
    ECR_REPOSITORY: "static-site"
    EKS_CLUSTER: "eksCluster"

```

Phase	What happens
pre_build	Finds your account ID, logs in to ECR, creates the static-site repository if it is missing.
build	Builds the image tagged with the build number and checks it exists locally.
post_build	Pushes to ECR, confirms the image is there, then deploys to EKS.

AWS_DEFAULT_REGION and CODEBUILD_BUILD_NUMBER are set by CodeBuild automatically. You do not define them. Commit and push all files to main.

Step 4: Create the CodeBuild project

1. Open **CodeBuild > Build projects > Create build project**. Name it devopsmatrix-build.
2. **Source**: GitHub, then connect and choose your repository. (You can also pick *CodePipeline* as the source and let Step 7 supply it.)
3. **Environment**: Managed image, Amazon Linux, Standard, latest image. Choose **New service role** and note its name.
4. Tick **Privileged** ("Enable this flag if you want to build Docker images"). **Without this, docker build fails.**
5. **Buildspec**: Use a buildspec file (default buildspec.yml). Create the project.

Do not start a build yet. The new role has no ECR or EKS permissions.

Step 5: Give the CodeBuild role permissions

Go to **IAM > Roles**, open the role from Step 4 (named like codebuild-devopsmatrix-build-service-role), then **Add permissions > Create inline policy > JSON**. Replace ACCOUNT_ID with your 12-digit account ID.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "ecr:CreateRepository",
        "ecr:DescribeRepositories",
        "ecr:DescribeImages",
        "ecr:BatchCheckLayerAvailability",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage",
        "ecr:BatchGetImage",
        "ecr:GetDownloadUrlForLayer"
      ],
      "Resource": "arn:aws:ecr:ap-south-1:<ACCOUNT_ID>:repository/static-site",
      "Effect": "Allow",
      "Action": "eks:DescribeCluster",
      "Resource": "arn:aws:eks:ap-south-1:<ACCOUNT_ID>:cluster/eksCluster"
    }
  ]
}

```

Step 6: Give the role access to the cluster

IAM permissions let CodeBuild *find* the cluster. Kubernetes has its own permission layer, so grant the role access there too. Run once from your laptop:

```
aws eks create-access-entry --cluster-name eksCluster --region ap-south-1 \
--principal-arn arn:aws:iam::<ACCOUNT_ID>:role/<CODEBUILD_ROLE_NAME>

aws eks associate-access-policy --cluster-name eksCluster --region ap-south-1 \
--principal-arn arn:aws:iam::<ACCOUNT_ID>:role/<CODEBUILD_ROLE_NAME> \
--policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy \
--access-scope type=cluster
```

If the role path is not /, for example /service-role/, use the ARN without the path. Without this step, kubectl get nodes fails with "Unauthorized". ClusterAdmin is fine for learning; narrow it later.

Step 7: Create the pipeline

1. Open **CodePipeline** > **Create pipeline**. Choose *Build custom pipeline*, name it devopsmatrix-pipeline, and let AWS create a new service role.
2. **Source**: GitHub (via GitHub App). Click *Connect to GitHub*, approve, choose the repo and branch main. Keep "Start the pipeline on source code change" on.
3. **Build**: Other build providers > AWS CodeBuild > select devopsmatrix-build.
4. **Test** and **Deploy**: choose *Skip* for both.
5. Review and **Create pipeline**. It starts a first run right away.

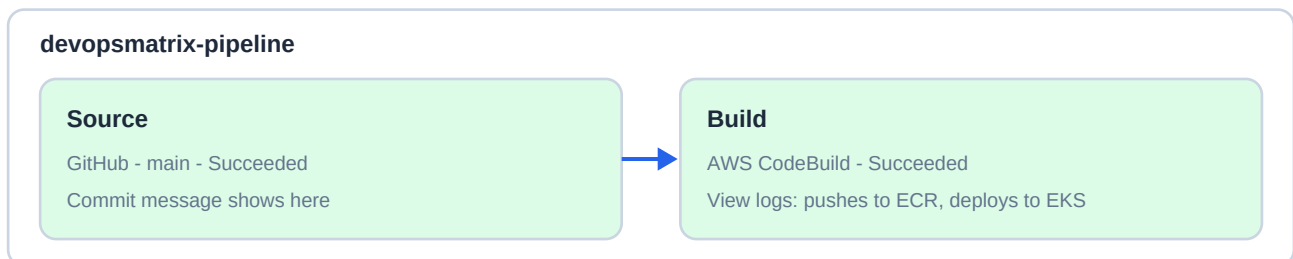


Figure 2. What a healthy run looks like (illustrative, not a live console screenshot).

Step 8: Run and verify

```
git commit -allow-empty -m "Trigger pipeline"
```

Open **CodePipeline** and watch Source, then Build, turn green. Click **View logs** on the Build stage to read the CodeBuild output. Then confirm the app:

```
aws eks update-kubeconfig --region ap-south-1 --name eksCluster
kubectl get pods,svc -n static-site
# open the EXTERNAL-IP / hostname of the service in a browser
```

In **ECR** > **static-site** you should see an image tagged with the build number (for example 1, 2, 3).

Troubleshooting

Symptom	Likely cause and fix
Source stage fails	GitHub connection is still Pending. Finish it under Developer Tools > Settings > Connections.
Cannot connect to the Docker daemon	Privileged mode is off in the CodeBuild environment (Step 4).
YAML_FILE_ERROR or buildspec not found	File is not named buildspec.yml or is not in the repo root. Use spaces, not tabs.

Symptom	Likely cause and fix
AccessDeniedException on ECR	The inline policy from Step 5 is missing an action, or the repository name in it is wrong.
kubectl get nodes: Unauthorized	Access entry not created, or created for the wrong role ARN (Step 6).
kubectl: command not found	The build image has no kubectl and deploy.sh did not install it. Use the version in Step 2.
Docker toomanyrequests on --pull	Docker Hub rate limit on the base image. Log in to Docker Hub in pre_build with credentials from Secrets Manager, or use an ECR-hosted base image.
Rollout times out	Pods cannot pull the image (check the node role has ECR read access) or the container crashes. Run kubectl describe pod and kubectl logs.
set image: container not found	The container in deployment.yaml is not named nginx.

Things to know

- **post_build runs even if build fails.** A failed build then shows a confusing push error. Scroll up in the logs to find the first real failure.
- The tag is the build number, which is unique but does not tell you the commit. Consider tagging with `${CODEBUILD_RESOLVED_SOURCE_VERSION}` (the commit SHA) so any image maps back to its code.
- To roll back, run `kubectl rollout undo deployment/static-site -n static-site`.

Next steps toward production

- Add tests and an image scan (ECR scan-on-push or Trivy) before the push.
- Add a staging environment and a manual approval action before production.
- Add resource limits and health probes to the Deployment.
- Define the pipeline, roles and ECR with CloudFormation, CDK or Terraform.